UNITÀ 6 · INTRODUZIONE AI SISTEMI OPERATIVI

User mode e kernel mode

Perché quando un gioco va in crash si chiude solo il gioco, mentre un errore nel posto sbagliato può fermare aerei e ospedali in tutto il mondo?

RISPONDI D'ISTINTO

Il programma che usi va in crash. Che cosa succede, secondo te?

SONDAGGIO

- **A** Si chiude solo quel programma
- **B** Si blocca tutto il computer
- **C** Dipende da che programma è
- **D** Si perdono tutti i file aperti, anche degli altri programmi

IL PROBLEMA

Che cosa potrebbe fare un programma senza regole?

Scrivere ovunque

Modificare la memoria di un altro programma, o del kernel stesso, per errore o per cattiveria.

Tenersi la CPU

Disattivare gli interrupt e non restituire mai il processore: lo scheduler non riprende più il controllo.

Parlare col disco

Mandare comandi diretti al controller e leggere i file di tutti gli utenti, password comprese.

> Il software da solo non basta a fermarli: serve un aiuto dell'**hardware**, cioè della CPU.

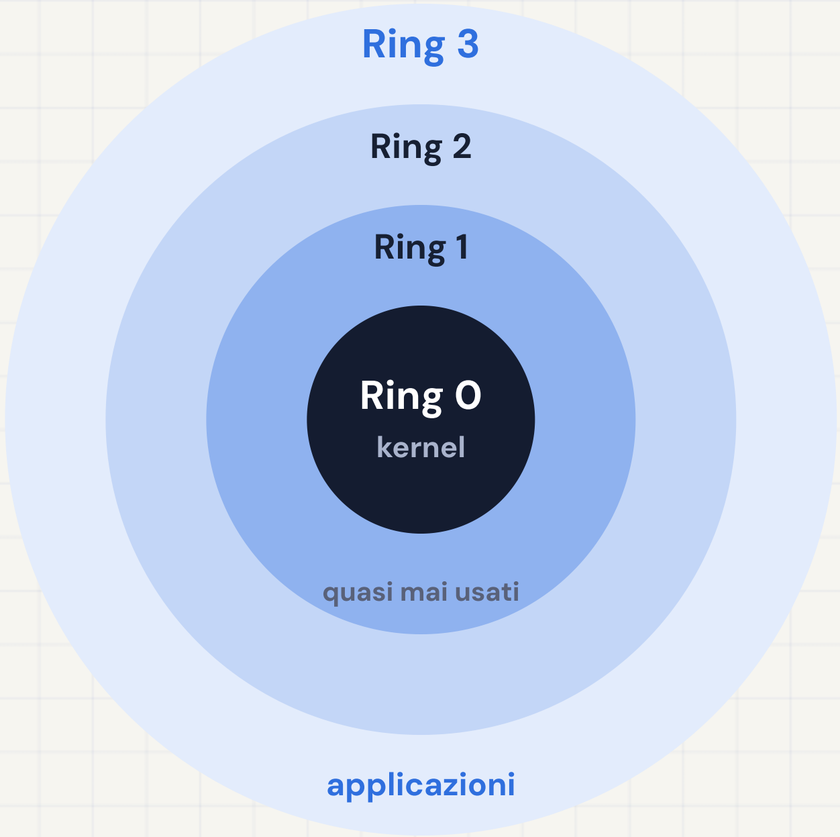
DUE MODALITÀ

La CPU sa sempre "chi sta guidando"

	User mode (modalità utente)	Kernel mode (modalità kernel)
Chi	Applicazioni, shell, servizi	Kernel e molti driver
Memoria	Solo la propria	Tutta
Istruzioni privilegiate	Vietate: la CPU genera un'eccezione	Permesse (I/O, interrupt, tabelle delle pagine)
Se sbaglia	Muore il processo	Si ferma il sistema

IL MODELLO A RING

Quattro anelli, due usati



Dal 1982, con l'80286, i processori Intel hanno **4 livelli di privilegio**: il ring 0 è il più potente, il ring 3 il più limitato.

Windows, Linux e macOS usano solo ring 0 per il kernel e ring 3 per le applicazioni.

L'idea degli anelli viene da Multics (anni '60–'70), che arrivò a 8 anelli.

QUIZ A TRABOCCHETTO

Quanti ring usano davvero Windows e Linux su un PC?

QUIZ

A 4, uno per ogni anello del processore

B 2: ring 0 e ring 3

C 1: il ring 0, per essere più veloci

D 8, come Multics

PUZZLE

Una chiamata di sistema, dall'inizio alla fine

1. La CPU torna in user mode e il programma riprende da dove era
2. La CPU passa in kernel mode e salta a un indirizzo scelto dal kernel
3. Il programma, in user mode, prepara la richiesta: "leggi 100 byte dal file"
4. Il kernel controlla che il programma abbia il permesso di leggere quel file
5. Esegue l'istruzione speciale di chiamata di sistema (su x86-64: syscall)
6. Il kernel legge i dati e li copia nella memoria del programma

AL COMPUTER · 12 minuti

Vedi il tempo del kernel

1. Apri Gestione attività (**Ctrl + Maiusc + Esc**), **Prestazioni**, **CPU**.
2. Clic destro sul grafico: attiva la voce che **mostra i tempi del kernel**. Compare una zona più scura.
3. Per 30 secondi non fare nulla. Poi copia una cartella con molti file piccoli (per esempio Immagini) su una chiavetta o in un'altra cartella.
4. Poi guarda un video in alta risoluzione per 30 secondi.
5. Annota in **tempi-kernel.odt** che cosa fa la zona scura nei tre momenti, con uno screenshot.

Controllo: la zona scura (tempo in kernel mode) cresce soprattutto durante la **copia dei file**: ogni file chiede aperture, letture e scritture, cioè chiamate di sistema e lavoro dei driver.

LA MEMORIA DIVISA IN DUE

Metà bassa all'utente, metà alta al kernel

Indirizzi virtuali (x86-64)	A chi
`FFFF FFFF FFFF FFFF` `FFFF 8000 0000 0000`	Kernel
`… vietati …`	Nessuno
`0000 7FFF FFFF FFFF` `0000 0000 0000 0000`	Processo utente

Calcola: quanti byte ha la parte utente?

Ultimo indirizzo + 1

$$0x7FFF\ FFFF\ FFFF + 1$$
$$= 0x8000\ 0000\ 0000$$
$$= 2^{47}\ \text{byte}$$

$2^{47} = 2^{7} \cdot 2^{40}$ = **128 TiB** per ogni processo.

AL COMPUTER · 15 minuti

Misura utente e kernel con PowerShell

1. Apri **Windows PowerShell** e scrivi il comando **M** (misura).
2. Esegui **C** (solo calcoli), poi di nuovo **M**.
3. Esegui **L** (legge migliaia di cartelle), poi di nuovo **M**.
4. In Calc calcola quanto cresce ogni tempo dopo C e dopo L. Salva **utente-kernel.ods**.

Controllo: dopo il ciclo cresce quasi solo il tempo **utente** (UserProcessorTime). Dopo la lettura delle cartelle cresce in modo evidente anche il tempo **privilegiato**, cioè in kernel mode.

```
M  Get-Process -Id $PID | Format-List UserProcessorTime, PrivilegedProcessorTime
C  for ($i=0; $i -lt 3000000; $i++) { }
L  Get-ChildItem C:\Windows -Recurse -Depth 2 -ErrorAction SilentlyContinue | Out-Null
```

MENTI CHE CI HANNO PRECEDUTO

Perché il kernel non si fida di nessuno

> **Il collaudo dei programmi può mostrare la presenza di errori, ma mai la loro assenza.**
>
> **Edsger W. Dijkstra**, informatico olandese, "Notes on Structured Programming", 1970

Se non si può dimostrare che un programma è senza errori, il sistema va costruito in modo che un errore **resti chiuso nel suo processo**.



QUANDO QUALCOSA VA STORTO

Due tipi di crash

Errore in user mode

"Il programma ha smesso di funzionare". La CPU segnala l'errore, il kernel **termina solo quel processo** e libera le sue risorse. Il resto continua.

Errore in kernel mode

Nessuno sopra il kernel può rimediare. Per non danneggiare i dati il sistema **si ferma**: schermata blu (Windows), kernel panic (Linux, macOS), riavvio.



METTITI ALLA PROVA

Un programma in user mode prova a disattivare gli interrupt. Che cosa succede?

QUIZ

A L'istruzione viene eseguita, ma più lentamente

B Il programma ottiene automaticamente i privilegi del kernel

C La CPU genera un'eccezione e il kernel, di solito, termina il programma

D Il computer si riavvia per sicurezza

AL COMPUTER · 12 minuti

Il diario degli incidenti del tuo PC

1. Premi **Win + R**, scrivi `perfmon /rel` e premi Invio: si apre **Monitoraggio affidabilità**.
2. Passa alla vista per **settimane**. Cerca le croci rosse: errori delle applicazioni e errori di Windows.
3. Clicca su 3 eventi e leggi il dettaglio: quale programma, quando, che cosa.
4. In Calc compila una tabella: data, programma, tipo (applicazione o Windows), **user mode o kernel mode?**
5. Salva come **incidenti-pc.ods**.

Controllo: la maggior parte degli errori riguarda **applicazioni** (user mode): si è chiuso un programma e il PC è andato avanti. Un arresto imprevisto di Windows, se c'è, è il segno di un problema in kernel mode o nell'hardware.

SFIDA FINALE

Apri il lucchetto con quello che hai imparato



- 1ª cifra: il ring in cui gira il kernel
- 2ª cifra: il ring in cui girano le applicazioni
- 3ª cifra: quanti anelli aveva Multics
- 4ª cifra: quanti ring offrono i processori x86

codice

BIGLIETTO D'USCITA

Completa prima di uscire

Le applicazioni girano in modalità _____.

Su x86 il kernel usa il ring _____.

Un programma entra nel kernel in modo ufficiale con una chiamata di _____.

Un errore in kernel mode su Windows produce la schermata _____.

0x8000 0000 0000 è uguale a 2 elevato alla ____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- La protezione ha bisogno dell'hardware: la CPU ha due modalità
- User mode: solo la propria memoria, niente istruzioni privilegiate
- Kernel mode: tutto è permesso, quindi ogni errore è grave
- x86 ha 4 ring; i SO usano il ring 0 e il ring 3
- Si entra nel kernel con chiamate di sistema, interrupt, eccezioni
- Crash di un'app: muore un processo. Crash del kernel: si ferma tutto

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.